CS-E4715 Supervised Machine Learning

Lecture 6: Support vector machines

Slides by Juho Rousu[2]

Adapted by Qi Chen[1,2], September 2026

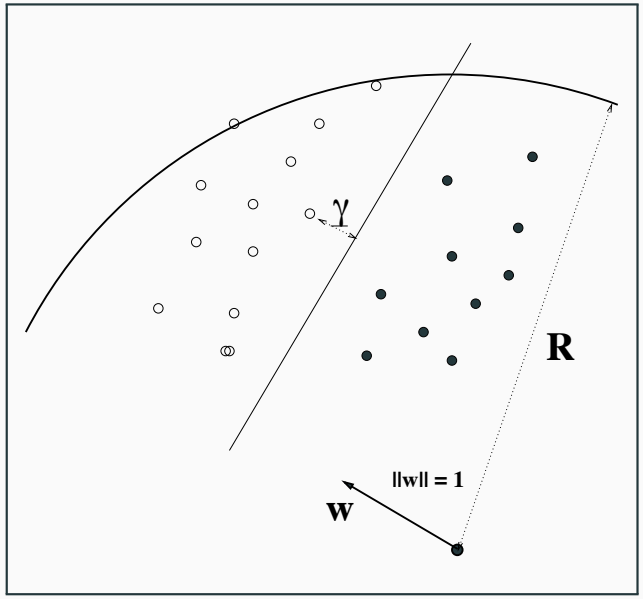
[1]ELLIS Institute Finland [2]Aalto CS

Finding optimal separating hyperplanes

Recall: Perceptron algorithm on linearly separable data

Recall the bound $(2R/\gamma)^2$ on the number of perceptron updates for linearly separable data.

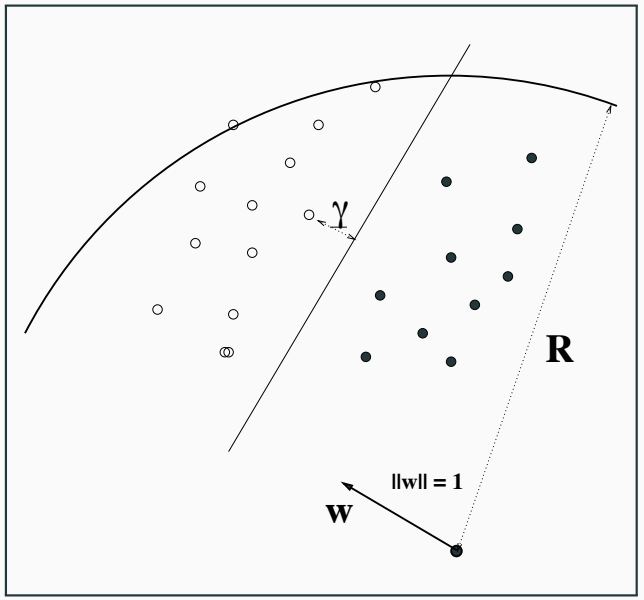
- γ: The largest achievable geometric margin in the training set, $\frac{y_i(\mathbf{w}^T\mathbf{x}_i+b)}{\|\mathbf{w}\|} \geq \gamma$ for all $i = 1\ldots, m$
- $R = \max_i \|\mathbf{x}_i\|$: The radius of the smallest ball centered at the origin that encloses the training data



The $(2R/\gamma)^2$ bound uses an affine separator and original-space R, γ via scaled augmentation, whereas $(R/\gamma)^2$ uses radius and margin in the homogeneous feature space.

Finding optimal separating hyperplanes

- The perceptron algorithm is guaranteed to find a consistent hyperplane if one exists.
- All training points lie on the correct side of the hyperplane.
- However, there are typically multiple consistent hyperplanes.
- Which one is best?



Maximum margin hyperplane

Convention: no intercept ($b = 0$) in this lecture. Appending a constant feature regularizes the intercept too; it changes the geometry.

One good solution is to choose the hyperplane $\mathbf{w}^T\mathbf{x} = 0$ that lies furthest away from the training data (maximizing the minimum geometric margin of the training examples):

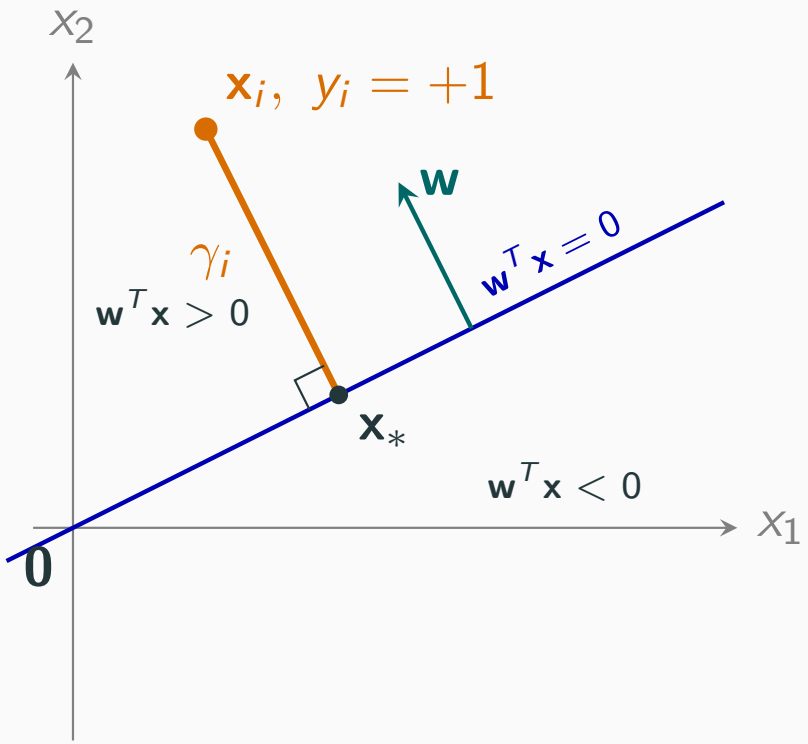
$$\begin{aligned} &\textit{Maximize } \gamma \\ &\quad w.r.t. \text{ variables } \mathbf{w} \in \mathbb{R}^d \\ &\textit{Subject to } \frac{y_i \mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|} \geq \gamma, \text{for all } i = 1, \ldots, m, \end{aligned}$$

The maximum margin hyperplane has good properties:

- Robustness: small change in the training data will not change the classifications too much
- Theoretically a large margin is tied to a low generalization error
- It can be found efficiently through incremental optimization

Support vector machines (SVM) are based on this principle

Functional margin and geometric margin



Functional margin: labeled score

$$m_i = y_i \mathbf{w}^T \mathbf{x}_i.$$

Geometric margin: signed distance

$$\gamma_i = \frac{y_i \mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|}, \qquad \boxed{m_i = \|\mathbf{w}\|\ \gamma_i}.$$

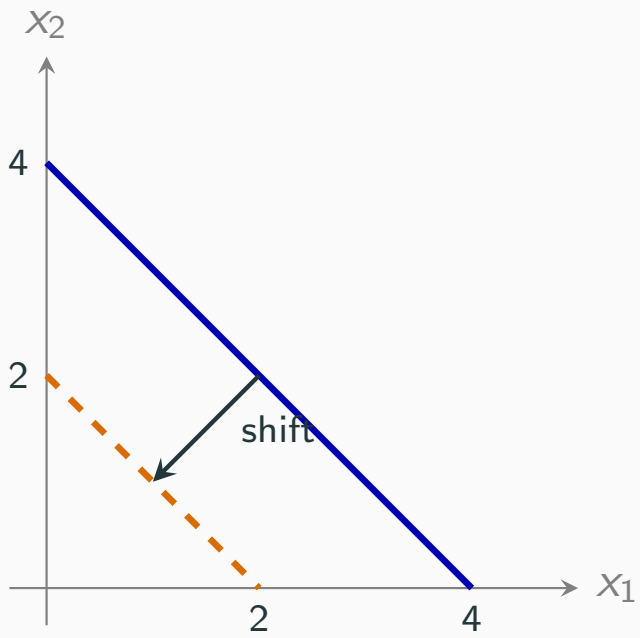
- $\gamma_i > 0$: correct side; $\gamma_i < 0$: wrong side.
- $|\gamma_i|$ is the perpendicular distance to the hyperplane.
- Functional margin is a score, not a distance in input space.

Rescale $\mathbf{w} \to c\mathbf{w}$ ($c > 0$): the hyperplane and γ_i stay the same, but $m_i \to c m_i$. For example, $\|\mathbf{w}\| = 2$, $\gamma_i = 1$ gives $m_i = 2$.

Rescaling with a bias: what changes?

Example: $\mathbf{w} = (1, 1)^T$, $b = -4$, and scale factor $c = 2$.

Scale only w; keep b fixed

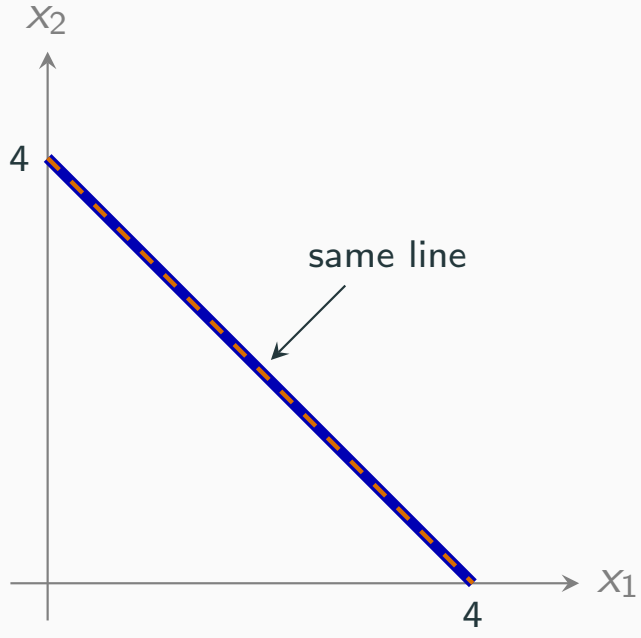


Original: $x_1 + x_2 - 4 = 0$

New: $2x_1 + 2x_2 - 4 = 0$

Same slope -1; different intercept.
Distance from origin: $2\sqrt{2} \to \sqrt{2}$.

Scale both w and b



Original: $x_1 + x_2 - 4 = 0$

New: $2x_1 + 2x_2 - 8 = 0$

Same slope and intercept.
Distance from origin stays $2\sqrt{2}$.

For $(\mathbf{w}, b) \to (c\mathbf{w}, cb)$ with $c > 0$: functional margins scale by c; geometric margins and predictions stay unchanged.

How to Maximize the Margin?

- However, the optimization problem

$$\begin{aligned} &\textit{Maximize } \gamma \\ &\quad w.r.t. \text{ variables } \mathbf{w} \in \mathbb{R}^d \\ &\textit{Subject to } \frac{y_i \mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|} \geq \gamma, \text{for all } i = 1, \ldots, m \end{aligned}$$

  does not give us a **unique** optimal weight vector $\mathbf{w}^*$
- This is because if $\mathbf{w}^*$ is a solution, then so is any vector $c\mathbf{w}^*, c > 0$ since

$$\frac{y_i (c\mathbf{w})^T \mathbf{x}_i}{\|c\mathbf{w}\|} = \frac{c y_i \mathbf{w}^T \mathbf{x}_i}{\sqrt{c^2 \mathbf{w}^T \mathbf{w}}} = \frac{c y_i \mathbf{w}^T \mathbf{x}_i}{c \|\mathbf{w}\|} = \frac{y_i \mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|}$$

- We can make the functional margin $y_i \mathbf{w}^T \mathbf{x}_i$ arbitrarily high just by scaling the norm of $\mathbf{w}$

How to Maximize the Margin?

- We could add a constraint $\|\mathbf{w}\| = 1$ to the optimization problem to get an unique answer.
- However, optimization would become more difficult to solve
- Instead, let us multiply the constraint on the geometric margin

$$\frac{y_i \mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|} \geq \gamma$$

by $\|\mathbf{w}\|$ to obtain a an equivalent constraint on the functional margin

$$y_i \mathbf{w}^T \mathbf{x}_i \geq \gamma \|\mathbf{w}\|$$

- Now fix the functional margin to 1: $\gamma \|\mathbf{w}\| = 1$ which gives $\gamma = \frac{1}{\|\mathbf{w}\|}$
- To maximize γ, we should minimize $\|\mathbf{w}\|$ with the constraint of having functional margin of at least 1

Support vector machine (SVM)

The so called hard margin support-vector machine (SVM, Cortes & Vapnik, 1995) solves the margin maximization as follows:

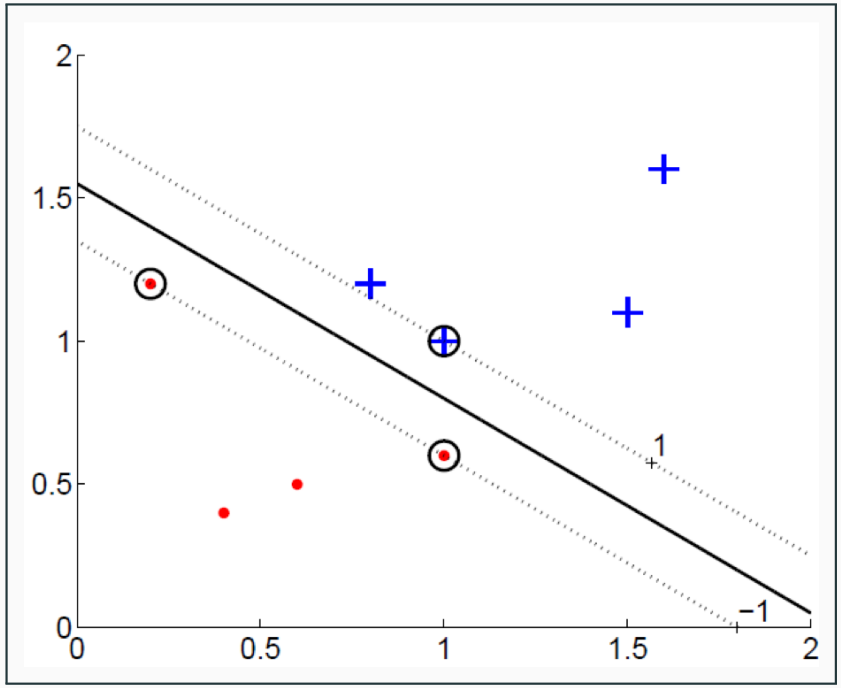
$$\begin{aligned} \textit{Minimize } & \frac{1}{2}\|\mathbf{w}\|^2 \\ w.r.t. \;\; & \text{variables } \mathbf{w} \in \mathbb{R}^d \\ \text{Subject to } & y_i \mathbf{w}^T \mathbf{x}_i \geq 1, \text{for all } i = 1, \ldots, m \end{aligned}$$

- We are minimizing the half of the squared norm of the weight vector, which gives the same answer as minimizing the norm, but easier to optimize
- This is equivalent of finding the maximal geometric margin over the same data

Geometrical interpretation

The maximum margin hyperplane separates the positive and negative examples with a minimum functional margin of 1

- In the hard-margin SVM, support vectors lie on the margin boundaries $y_i \mathbf{w}^T \mathbf{x}_i = 1$ and determine the optimal hyperplane. A precise definition will follow in the dual formulation.
- Adding or removing points strictly outside the margin ($y\mathbf{w}^T\mathbf{x} > 1$) leaves the hard-margin optimum unchanged.



Generalization: margin and model complexity

- Consider the class of real-valued linear scores

$$\mathcal{F}_B = \{f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} : \|\mathbf{w}\| \leq B\}, \qquad \|\mathbf{x}\| \leq R.$$

- Its empirical Rademacher complexity satisfies

$$\boxed{\widehat{\mathfrak{R}}_S(\mathcal{F}_B) \leq \frac{BR}{\sqrt{m}}.}$$

- A smaller norm budget B means lower complexity: the class has less ability to fit random signs.
- **SVM balances fit and complexity.** A small norm together with small training hinge loss gives a tighter generalization bound.

Reminder: When $\min_i y_i \mathbf{w}^T\mathbf{x}_i = 1$, the geometric margin is $\gamma = 1/\|\mathbf{w}\|$, so a larger margin corresponds to a smaller norm.

Non-separable data

The so called hard margin support-vector machine assumes linearly separable data

$$\begin{aligned} \textit{Minimize } & \frac{1}{2}\|\mathbf{w}\|^2 \\ w.r.t. & \text{ variables } \mathbf{w} \in \mathbb{R}^d \\ \text{Subject to } & y_i \mathbf{w}^T \mathbf{x}_i \geq 1, \text{for all } i = 1, \ldots, m \end{aligned}$$

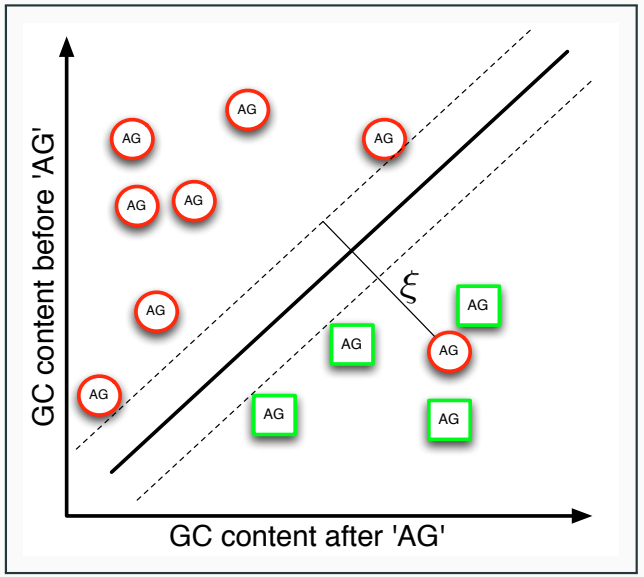
- In the non-separable case, for any hyperplane, there will be an example with a non-positive margin $y_i \mathbf{w}^T \mathbf{x}_i \leq 0$ which violates the constraint $y_i \mathbf{w}^T \mathbf{x}_i \geq 1$
- Our optimization problem has no feasible solution
- We need to extend our model to allow misclassified training points

Non-separable data

- To allow non-separable data, we allow the functional margin of some data points to be smaller than 1 by a slack variable $\xi_i \geq 0$
- The relaxed margin constraint will be expressed as

$$y_i \mathbf{w}^T \mathbf{x}_i \geq 1 - \xi_i, \xi_i \geq 0$$

- $\xi_i = 0$ corresponds to having large enough margin (≥ 1)
- For the smallest feasible slack, $\xi_i > 1$ corresponds to a negative margin (a misclassified point).
- Support vectors lie on or inside the margin boundaries.



Soft-Margin SVM (Cortes & Vapnik, 1995)

The soft-margin SVM allows non-separable data by using the relaxed the margin constraints

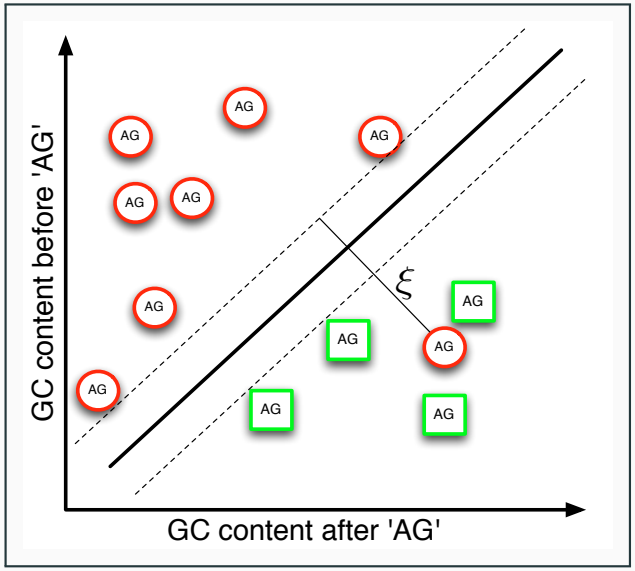
$$Minimize \frac{1}{2}||\mathbf{w}||^2 + \frac{C}{m}\sum_{i=1}^{m}\xi_i$$

$$w.r.t \text{ variables } \mathbf{w}, \boldsymbol{\xi}$$

$$\text{Subject to } y_i \mathbf{w}^T \mathbf{x}_i \geq 1 - \xi_i$$

$$\text{for all } i = 1, \ldots, m.$$

$$\xi_i \geq 0, \text{for all } i = 1, \ldots, m.$$



- The sum (or average) of slack variables appear as a penalty in the objective
- The coefficient $C > 0$ controls the balance between model complexity (low C) and empirical error (high C)

The loss function in SVM

- We can interpret the soft-margin SVM in terms of minimization of a loss function
- Observe the relaxed margin constraint:

$$y_i \mathbf{w}^T \mathbf{x}_i \geq 1 - \xi_i, \xi_i \geq 0$$

- By rearranging, the same can be expressed as

$$\xi_i \geq 1 - y_i \mathbf{w}^T \mathbf{x}_i, \xi_i \geq 0$$

  and further

$$\xi_i \geq \max(1 - y_i \mathbf{w}^T \mathbf{x}_i, 0)$$

- The right-hand side is so called Hinge loss:

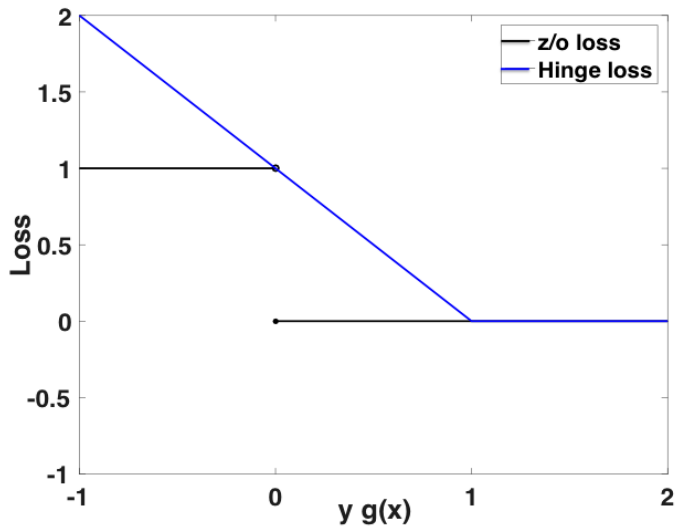
$$L_{Hinge}(y, \mathbf{w}^T \mathbf{x}) = \max(1 - y \mathbf{w}^T \mathbf{x}, 0)$$

Loss functions: Hinge loss

Hinge loss can be written for $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ as

$$L_{Hinge}(y, f(\mathbf{x})) = \max(1 - yf(\mathbf{x}), 0)$$

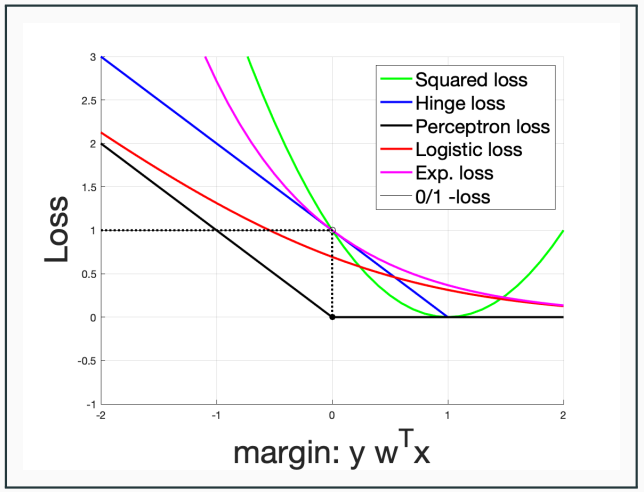
- Hinge loss is a convex upper bound of zero-one loss
- Hinge loss is zero if margin $y_i f(\mathbf{x}) \geq 1$
- For a misclassified example, margin is negative and Hinge loss is $\mathcal{L}_{Hinge}(f(\mathbf{x}), y_i)) > 1$
- The loss grows linearly in the margin violation $1 - yf(\mathbf{x})$, for margins < 1



Interlude: Convex loss functions for classification

We have so far seen three different convex loss functions for classification:

- Perceptron loss
- Hinge loss
- Logistic loss



In addition there are multiple other convex loss functions:

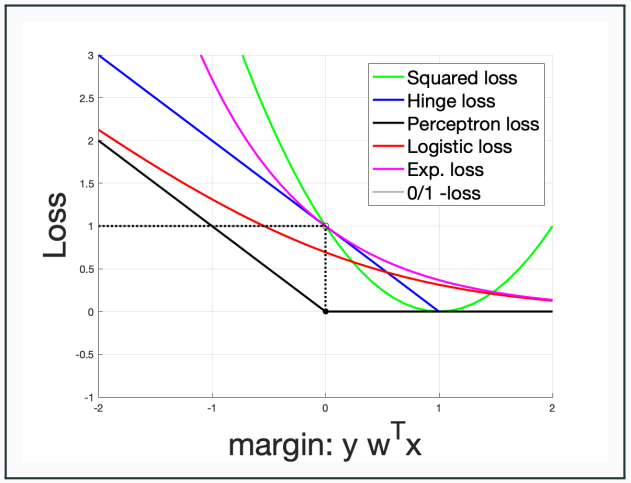
- Squared loss - used for regression, not optimal for classification
- Exponential loss - used in boosting

Because of convexity, all of the functions support fast optimization through gradient based approaches, unlike the zero-one loss

Interlude: Convex loss functions for classification

- Three of the loss functions in the figure are not only convex, but also upper-bounds of zero-one loss: Exponential loss, Hinge loss, Squared loss

- Logistic loss can be scaled by a constant to make it an upper bound of zero-one loss
- Perceptron loss is not an upper bound of zero-one loss



- The benefit of being an upper bound of zero-one loss: if we minimize the upper bound, the zero-one loss will also usually be small
- Because of the convexity, minimization of the upper bound can be done fast, unlike minimization of zero-one loss

Optimization

Quadratic programming

The soft-margin SVM corresponds to a Quadratic program (QP)

$$\begin{aligned} \textit{Minimize} \frac{1}{2}\|\mathbf{w}\|^2 + \frac{C}{m}\sum_{i=1}^{m}\xi_i \\ w.r.t \text{ variables } \mathbf{w}, \boldsymbol{\xi} \\ \text{Subject to } y_i \mathbf{w}^T \mathbf{x}_i \geq 1 - \xi_i \\ \text{for all } i = 1, \ldots, m. \\ \xi_i \geq 0, \text{for all } i = 1, \ldots, m. \end{aligned}$$

- This SVM problem is a convex quadratic program with a unique optimal weight vector $\mathbf{w}$.
- The QP objective is a quadratic function of the variables
- The QP constraints are linear functions of the variables
- When data is small, QP solvers in optimization libraries can be used to solve the soft-margin SVM problem

Soft-margin SVM as a regularised learning problem

- We can rewrite the soft-margin SVM problem

$$\begin{aligned} \textit{Minimize} \quad & \tfrac{1}{2}||\mathbf{w}||^2 + \tfrac{C}{m}\textstyle\sum_{i=1}^{m} \xi_i \\ \text{Subject to} \quad & \xi_i \geq \max(1 - y_i \mathbf{w}^T \mathbf{x}_i, 0) \\ & \text{for all } i = 1, \ldots, m. \\ & \xi_i \geq 0 \end{aligned}$$

  equivalently in terms of Hinge loss as

$$\min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}_{Hinge}(\mathbf{w}^T \mathbf{x}_i, y_i) + \frac{\lambda}{2}||\mathbf{w}||^2$$

- This is a so called **regularized learning problem**
  - First term minimizes a loss function on training data
  - Second term, called the **regularizer**, controls the complexity of the model
  - The parameter $\lambda = \frac{1}{C}$ controls the balance between the two terms

Half-time poll: Effect of Coefficient C in Soft Margin SVM

Imagine you're training a soft margin Support Vector Machine (SVM) with the following objective function:

$$\min_{w,\xi} \ \frac{1}{2}\|w\|^2 + \frac{C}{m}\sum_{i=1}^{m}\xi_i$$

where ξ_i is the slack variable for data point i.

If you set the value of C to be **very low**, which of the two following linear models would you expect to obtain?

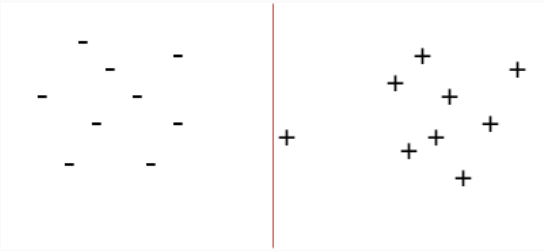


Figure 1: Picture One

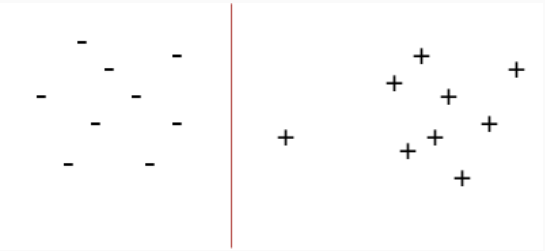


Figure 2: Picture Two

Optimization on big data

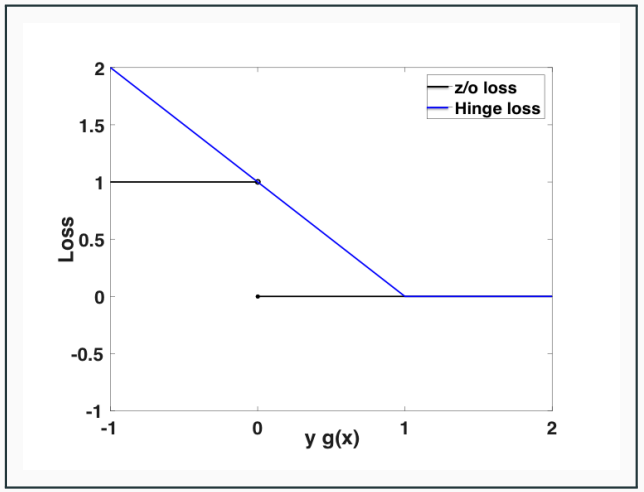
On big data, a stochastic gradient descent procedure is a good option

Rewrite the regularized learning problem as an average:

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m} J_i(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m} \left(\mathcal{L}_{Hinge}(\mathbf{w}^T\mathbf{x}_i, y_i) + \frac{\lambda}{2}||\mathbf{w}||^2\right)$$

where $J_i(\mathbf{w}) = L_{Hinge}(\mathbf{w}^T\mathbf{x}_i, y_i) + \frac{\lambda}{2}||\mathbf{w}||^2$

However, Hinge loss is not differentiable at 1 (because of the 'Hinge' at 1), so cannot simply compute the gradient $\nabla J_i(\mathbf{w})$



Subgradients

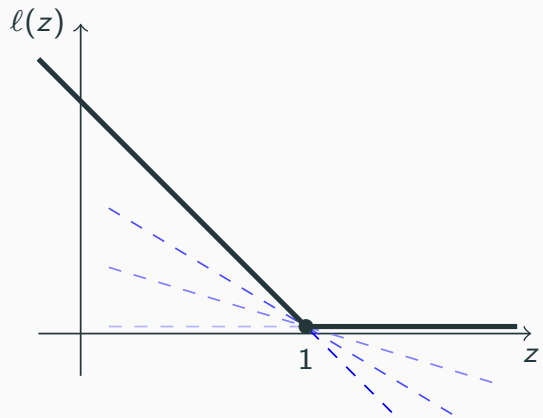
Definition (Subgradient)
Let $f : \mathbb{R}^d \to \mathbb{R}$ be convex. A vector $\mathbf{g}$ is a **subgradient** of f at $\mathbf{w}$ if

$$f(\mathbf{v}) \geq f(\mathbf{w}) + \mathbf{g}^T(\mathbf{v} - \mathbf{w}) \quad \text{for all } \mathbf{v}$$

i.e. the linear function through $(\mathbf{w}, f(\mathbf{w}))$ with slope $\mathbf{g}$ lies below f everywhere.

- The set of all subgradients is the **subdifferential** $\partial f(\mathbf{w})$
- If f is differentiable at $\mathbf{w}$: $\partial f(\mathbf{w}) = \{\nabla f(\mathbf{w})\}$
- At a kink there are many, e.g. for the hinge $\ell(z) = \max(0, 1 - z)$:

$$\partial \ell(z) = \begin{cases} \{-1\} & z < 1 \\ [-1, 0] & z = 1 \\ \{0\} & z > 1 \end{cases}$$



Dashed: subgradients at the kink

- **Subgradient descent**: use any $\mathbf{g} \in \partial f(\mathbf{w})$ in place of $\nabla f(\mathbf{w})$; for convex f with suitable stepsizes it still converges

Subgradients of the Hinge loss

- We can differentiate the linear pieces of the loss separately

$$L_{Hinge}(\mathbf{w}^T\mathbf{x}_i, y_i) = \begin{cases} 1 - y_i\mathbf{w}^T\mathbf{x}_i, & \text{if } y_i\mathbf{w}^T\mathbf{x}_i < 1 \\ 0, & \text{if } y_i\mathbf{w}^T\mathbf{x}_i \geq 1 \end{cases}$$

- Choose a subgradient of the hinge loss:

$$g_i(\mathbf{w}) = \begin{cases} -y_i\mathbf{x}_i & \text{if } y_i\mathbf{w}^T\mathbf{x}_i < 1 \\ \mathbf{0} & \text{if } y_i\mathbf{w}^T\mathbf{x}_i \geq 1 \end{cases}$$

- At $y_i\mathbf{w}^T\mathbf{x}_i = 1$, choose the zero subgradient of hinge loss.
- This is justified: at $y_i\mathbf{w}^T\mathbf{x}_i = 1$ the subdifferential of the hinge loss w.r.t. $\mathbf{w}$ is $\{-t\, y_i\mathbf{x}_i : t \in [0, 1]\}$, which contains $\mathbf{0}$

Stochastic gradient descent algorithm for SVM

To find the update direction we express $J_i(\mathbf{w})$ as a piecewise differentiable function

$$J_i(\mathbf{w}) = L_{Hinge}(\mathbf{w}^T\mathbf{x}_i, y_i) + \frac{\lambda}{2}||\mathbf{w}||^2 = \begin{cases} 1 - y_i\mathbf{w}^T\mathbf{x}_i + \frac{\lambda}{2}||\mathbf{w}||^2, & \text{if } y_i\mathbf{w}^T\mathbf{x}_i < 1 \\ 0 + \frac{\lambda}{2}||\mathbf{w}||^2 & \text{if } y_i\mathbf{w}^T\mathbf{x}_i \geq 1 \end{cases}$$

A valid choice of subgradient (including at the hinge) is:

$$g_i(\mathbf{w}) = \begin{cases} -y_i\mathbf{x}_i + \lambda\mathbf{w} & \text{if } y_i\mathbf{w}^T\mathbf{x}_i < 1 \\ \mathbf{0} + \lambda\mathbf{w} & \text{if } y_i\mathbf{w}^T\mathbf{x}_i \geq 1 \end{cases}$$

Update direction is the negative chosen subgradient $-g_i(\mathbf{w})$

Stochastic gradient descent algorithm for soft-margin SVM

Initialize $\mathbf{w} = 0$; $t = 1$

repeat

Draw a training example $(\mathbf{x}, y)$ uniformly at random

Choose a subgradient g_t for the sampled example:

$$g_t = \begin{cases} -y\mathbf{x} + \lambda\mathbf{w} & \text{if } y\mathbf{w}^T\mathbf{x} < 1 \\ \lambda\mathbf{w} & \text{if } y\mathbf{w}^T\mathbf{x} \geq 1 \end{cases}$$

Determine a stepsize η_t

Update $\mathbf{w} = \mathbf{w} - \eta_t g_t$

$t = t + 1$

until stopping criterion satisfied

Output $\mathbf{w}$

- For the stepsize, diminishing stepsize of $\eta_t = \eta_0/t$, can be used, where $\eta_0 > 0$ is a hyperparameter giving the initial stepsize
- As the stopping criterion, one can use, e.g. the relative improvement of the objective between two successive iterations

Interpreting the update

$$\mathbf{w} = \mathbf{w} - \eta_t \left(\lambda \mathbf{w} + \begin{cases} -y_i \mathbf{x}_i & \text{if } y_i \mathbf{w}^T \mathbf{x}_i < 1 \\ \mathbf{0} & \text{otherwise} \end{cases} \right)$$

- Each update shrinks the weight vector by a factor $(1 - \eta_t \lambda) \implies$ pushes towards a wider margin $1/\|\mathbf{w}\|$ and adds regularization
- If the example has positive Hinge loss (functional margin < 1), we add $\eta_t y_i \mathbf{x}_i$ to the weight vector
- This increases the functional margin of that example by $\eta_t \|\mathbf{x}_i\|^2$, i.e. decreases its Hinge loss, similarly to the perceptron update

Interpreting the update

$$\mathbf{w} = \mathbf{w} - \eta_t \left(\lambda \mathbf{w} + \begin{cases} -y_i \mathbf{x}_i & \text{if } y_i \mathbf{w}^T \mathbf{x}_i < 1 \\ \mathbf{0} & \text{otherwise} \end{cases} \right)$$

- Compare to the perceptron update:

$$\mathbf{w} = \mathbf{w} + \begin{cases} y_i \mathbf{x}_i & \text{if } y_i \mathbf{w}^T \mathbf{x}_i \leq 0 \\ \mathbf{0} & \text{otherwise} \end{cases}$$

- Both share the idea of adding to $\mathbf{w}$ the training example multiplied by the label $y_i \mathbf{x}_i$, SVM does this to all examples that have too small margin, not only misclassified ones
- The regularization term multiplies $\mathbf{w}$ by $1 - \eta_t \lambda$ at every iteration.

Interpreting the update

- Consider the evolution of the weight vector $\mathbf{w}^{(t)}$ by the stochastic gradient optimization
- Assume $\lambda = 0$ and that $(\mathbf{x}^{(t)}, y^{(t)})$ is the t'th training example drawn by the algorithm that has positive Hinge loss, and $\eta^{(t)}$ is the learning rate
- Then we have

$$\begin{aligned}
\mathbf{w}^{(1)} &= \eta_1 y^{(1)} \mathbf{x}^{(1)} \\
\mathbf{w}^{(2)} &= \eta_1 y^{(1)} \mathbf{x}^{(1)} + \eta_2 y^{(2)} \mathbf{x}^{(2)} \\
\mathbf{w}^{(3)} &= \eta_1 y^{(1)} \mathbf{x}^{(1)} + \eta_2 y^{(2)} \mathbf{x}^{(2)} + \eta_3 y^{(3)} \mathbf{x}^{(3)} \\
\mathbf{w}^{(t)} &= \sum_{j=1}^{t} \eta_j y^{(j)} \mathbf{x}^{(j)}
\end{aligned}$$

- Thus the weight vector is a linear combination of the training examples that have been updated on so far

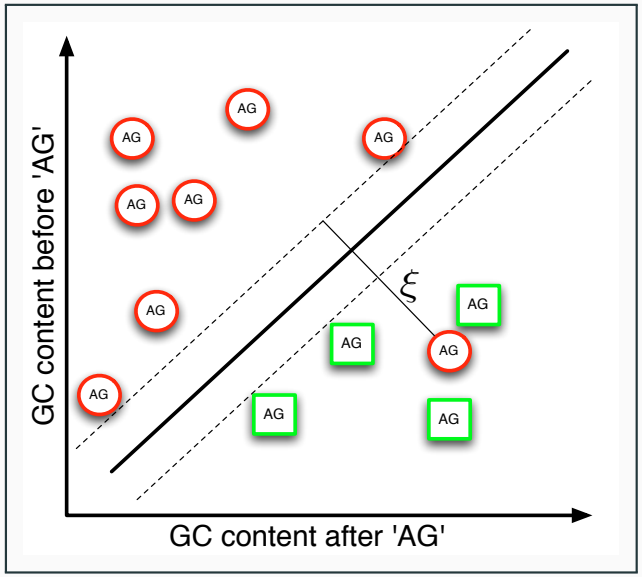
Dual soft-margin SVM

Dual representation of the optimal hyperplane

It can be shown theoretically that the **optimal hyperplane** of the soft-margin SVM has a **dual representation** as the linear combination of the training data

$$\mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

- The coefficients, also called the **dual variables** are non-negative $\alpha_i \geq 0$
- The positive coefficients $\alpha_i > 0$ appear if and only if $\mathbf{x}_i$ is a support vector, for other training points we have $\alpha_i = 0$



Dual representation of the optimal hyperplane

- Consequently, the functional margin $y\mathbf{w}^T\mathbf{x}$ also can be expressed using the support vectors:

$$y\mathbf{w}^T\mathbf{x} = y\sum_{i=1}^{m}\alpha_i y_i \mathbf{x}_i^T\mathbf{x}$$

- The norm of the weight vector can be expressed as

$$\mathbf{w}^T\mathbf{w} = \sum_{i=1}^{m}\alpha_i y_i \mathbf{x}_i^T \sum_{j=1}^{m}\alpha_j y_j \mathbf{x}_j = \sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$$

- Note that the training data appears in pairwise inner products: $\mathbf{x}_i^T\mathbf{x}_j$

Dual representations

- We can replace the explicit inner products with a kernel function

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$$

  which computes an inner product in the space of the arguments, here $\mathbb{R}^d$
- Plug in:
  - Margin:

$$y\mathbf{w}^T\mathbf{x} = y \sum_{i=1}^{m} \alpha_i y_i \kappa(\mathbf{x}_i, \mathbf{x})$$

  - Squared norm:

$$\|\mathbf{w}\|^2 = \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)$$

Dual Soft-Margin SVM

A dual optimization problem for the soft-margin SVM with kernels is given by

$$\begin{aligned} \textit{Maximize} \quad & OBJ(\boldsymbol{\alpha}) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \\ \textit{w.r.t} \quad & \text{variables } \boldsymbol{\alpha} \in \mathbb{R}^m \\ \text{Subject to} \quad & 0 \leq \alpha_i \leq C/m \\ & \text{for all } i = 1, \ldots, m \end{aligned}$$

- The dual is a concave quadratic maximization; optimal $\boldsymbol{\alpha}$ need not be unique, although the primal $\mathbf{w}$ is unique.
- At optimum, $\mathbf{w} = \sum_i \alpha_i y_i \phi(\mathbf{x}_i)$ in feature space; for a linear kernel, $\phi(\mathbf{x}) = \mathbf{x}$.
- The data only appears through the kernel function $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- Full derivation requires techniques of optimization theory, which we will skip here

Kernel trick

- We can consider transformations of the input with some feature map $\phi : \mathbb{R}^d \mapsto \mathbb{R}^k$ to a feature space $\mathcal{F} = \mathbb{R}^k$
- The optimal hyperplane $\mathbf{w} \in \mathbb{R}^k$ will satisfy: $\mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \phi(\mathbf{x}_i)$
- Assume κ_ϕ computes an inner product in the feature space $\kappa_\phi(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$
- Then can compute the hyperplane in the transformed space

$$\mathbf{w}^T \phi(\mathbf{x}) = \sum_{i=1}^{m} \alpha_i y_i \kappa_\phi(\mathbf{x}_i, \mathbf{x})$$

  and the squared norm of the weight vector $\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w} = \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \kappa_\phi(\mathbf{x}_i, \mathbf{x}_j)$
- We do not need to explicitly refer to the feature vectors $\phi(\mathbf{x})$ or the weight vector $\mathbf{w}$, both of which could be high-dimensional
- This is sometimes called the **kernel trick**

Stochastic Dual Coordinate Ascent

Stochastic Dual Coordinate Ascent for dual SVM

- Consider an algorithm updating one randomly selected dual variable (i.e. coordinate, hence the name of the method) α_i at a time , while keeping the other dual variables fixed
- We take the direction of the **positive gradient** of the dual SVM objective $OBJ(\boldsymbol{\alpha})$

$$\Delta\alpha_i = \frac{\partial}{\partial\alpha_i} OBJ(\boldsymbol{\alpha}) = \frac{\partial}{\partial\alpha_i}\left(\sum_{k=1}^{m}\alpha_k - \frac{1}{2}\sum_{k=1}^{m}\sum_{j=1}^{m}\alpha_k\alpha_j y_k y_j \kappa(\mathbf{x}_k, \mathbf{x}_j)\right)$$
$$= 1 - y_i\sum_{j=1}^{m}\alpha_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) = 1 - y_i f(\mathbf{x}_i)$$

where we used the dual representation
$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = \sum_{j=1}^{m}\alpha_j y_j \kappa(\mathbf{x}_j, \mathbf{x})$

Stochastic Dual Coordinate Ascent for dual SVM

- The update direction thus depends on the margin:

$$\Delta\alpha_i = \begin{cases} < 0 & \text{if } y_i f(\mathbf{x}_i) > 1 \\ 0 & \text{if } y_i f(\mathbf{x}_i) = 1 \\ > 0 & \text{if } y_i f(\mathbf{x}_i) < 1 \end{cases}$$

- If the margin is too small (Hinge loss is positive), α_i is increased, if there is more than the required margin, α_i is decreased
- Note the analogy to updating $\mathbf{w} = \mathbf{w} + \eta_t y \mathbf{x}$, when $\mathbf{x}$ has too small margin : η_t and α_i have similar roles

Stepsize

- We can easily find the optimal update direction and step-size by setting

$$\frac{\partial}{\partial \alpha_i} OBJ(\boldsymbol{\alpha}) = 0,$$

  it will give:

$$\alpha_i = \frac{1 - y_i \sum_{j \neq i} \alpha_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)}{\kappa(\mathbf{x}_i, \mathbf{x}_i)}$$

- Finally, the bounds $0 \leq \alpha_i \leq C/m$ need to be adhered $\alpha_i = \min(C/m, \max(\alpha_i, 0))$

Stepsize

Assume $\kappa(\mathbf{x}_i, \mathbf{x}_i) > 0$. For a PSD kernel, if it is zero, the coordinate objective is linear and maximized at $\alpha_i = C/m$.

- We can easily find the optimal update direction and step-size by setting

$$\frac{\partial}{\partial \alpha_i} OBJ(\boldsymbol{\alpha}) = 0,$$

  it will give:

$$\alpha_i = \frac{1 - y_i \sum_{j \neq i} \alpha_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)}{\kappa(\mathbf{x}_i, \mathbf{x}_i)}$$

- Finally, the bounds $0 \leq \alpha_i \leq C/m$ need to be adhered $\alpha_i = \min(C/m, \max(\alpha_i, 0))$

Stochastic Dual Coordinate Ascent for SVM

Initialize $\boldsymbol{\alpha} = \mathbf{0}$

repeat

Select a random training example (x_i, y_i)

Update the dual variable: $\alpha_i = \frac{1 - y_i \sum_{j \neq i} \alpha_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)}{\kappa(\mathbf{x}_i, \mathbf{x}_i)}$

Clip to satisfy the constraints: $\alpha_i = \min(C/m, \max(0, \alpha_i))$

until stopping criterion is satisfied

return $\boldsymbol{\alpha}$

Stochastic Dual Coordinate Ascent for SVM

For $\kappa(\mathbf{x}_i, \mathbf{x}_i) = 0$, set $\alpha_i = C/m$; otherwise use the update below.

Initialize $\boldsymbol{\alpha} = \mathbf{0}$

repeat

Select a random training example (x_i, y_i)

Update the dual variable: $\alpha_i = \frac{1 - y_i \sum_{j \neq i} \alpha_j y_j \kappa(\mathbf{x}_i, \mathbf{x}_j)}{\kappa(\mathbf{x}_i, \mathbf{x}_i)}$

Clip to satisfy the constraints: $\alpha_i = \min(C/m, \max(0, \alpha_i))$

until stopping criterion is satisfied

return $\boldsymbol{\alpha}$

Summary

- Support vector machines are classification methods based on the principle of margin maximization
- SVMs can be efficiently optimized using Stochastic gradient techniques specially developed for piecewise differentiable functions, such as the higge loss
- Dual representation of SVM allows the use of kernel functions (more on kernels next lecture)